

AsymptoticAnalysis

Precision boundaries, native compatibility, and the Wolfram Language comparison

An incremental audit after the two archived review waves

Repository	VladimirReshetnikov/Asymptotic
Pinned revision	6687962f3c858a4f93623cfc496f33e35c6763d4
Commit time	September 10, 2026, 00:18:17 UTC
Local date	September 9, 2026, 17:18:17 PDT
Review basis	Current source, maintained finding register, both wave indexes, selected original finding ledgers, and official Wolfram documentation
Execution	Independent exact Python models and patch fixtures executed; native Wolfram package and candidate patch not executed

Principal conclusion. The most useful additional work is at the boundary where a returned native Taylor series becomes a precision-bearing package object. One observable path does not verify that the native result actually contains all coefficients needed for the requested cutoff. An honest, shorter Taylor oracle can therefore be interpreted as having zero unknown coefficients. This report gives the exact countermodel, a minimal coefficient-demand theorem, a fail-closed patch candidate, and focused regression specifications.

Two further deltas make the existing capability and compatibility discussions more concrete: unsupported coefficient queries on valid forward objects reach inverse-only metadata before declining, and blanket protection of conditional sources obstructs the stated automatic native-coverage objective. A separate UI proposal concerns the deliberately syntactic meaning of the native Computed status. These are not presented as four independently reproduced native bugs.

Delivery. The accompanying archive includes this article in TeX and PDF, original audit helpers, desired-contract Wolfram tests, a native observation script, an exact Python oracle, the novelty ledger, and execution records. No checksum files are included.

Executive assessment

The repository is no longer well characterized as just a replacement for `InverseSeries`. It combines a sparse real power–log calculus, specialized inverse constructions, exact carriers and other adapted scales, numerical inverse checks, a rational interval certificate path, and a distinct wrapper for native `Series/Asymptotic` results. The explicit native wrapper preserves the native result without inventing a real analytic remainder. That separation is a strength to retain, not a missing feature to propose again.[2, 3, 5, 4, 16]

The archive already contains extensive advice about sparse export, nonlinear logarithmic frontiers, assumptions, parameter dependence, resource limits, refinement, testing, and future transseries functionality. This report does not reissue that backlog. Its priority is to identify a small number of additional, checkable boundary obligations and to state precisely how they differ from what was previously recorded.[10, 11, 12]

ID	Priority	Additional contribution and evidence status
T01	High if reached	Observable Taylor ingress ignores the returned endpoint. The source mechanism and a truthful analytic countermodel are established; public Wolfram execution remains unverified.
A01	Low–medium	A valid forward object can enter the inverse coefficient object overload without the required model. This sharpens the capability issue into a specific message-free refusal contract; native diagnostics are predicted, not observed.
N01	Medium for coverage	A conditional complex polynomial isolates an obstruction in the protected-source routing policy. It is an incompleteness relative to the accepted coverage goal, not a violation of the documented current routing policy.
U01	Low; proposal	<code>NativeEvaluationStatus->"Computed"</code> means only that no native call remains. A proposed outcome classification would distinguish failure markers without claiming analytic correctness.

The repair recommendation is deliberately small. First, reject insufficient native Taylor order in the ordinary observable path. Then add a model-capability guard before inverse-only metadata access. Treat routing changes as a separate compatibility decision with explicit condition-preservation tests. Do not turn native formal output into an analytic object merely to make more tests pass.

Validation boundary. The included Python program completed 940 exact model and synthetic patch-fixture checks. Of these, 868 are finite rational coefficient-demand cases; the count is not 940 independent package behaviors. There was no successful Wolfram kernel invocation, package run, patch run, or native timing measurement in this audit. The Wolfram service returned connection errors. Historical and repository-maintained native results remain evidence for their own recorded snapshots and suites, not for the new candidate patch.

Contents

Executive assessment	1
1 Scope, evidence, and the nonduplication boundary	4
1.1 A reproducible revision rather than a moving branch	4
1.2 What was used to exclude earlier findings	4
1.3 Evidence classes used below	5
2 Architecture and mathematical contracts	5
2.1 The positive local coordinate is part of the result	5
2.2 Ordinary inversion and specialized inverse families	5
2.3 Three different meanings of checking an inverse	6
2.4 Native delegation is a different contract, not a shortcut proof	6
3 Comparison with built-in Wolfram Language	7
3.1 Compare workloads and evidence, not function names	7
3.2 An example where a fair comparison is straightforward	8
3.3 Where the package is genuinely differentiated	8
3.4 Native capability is not the same as native proof strength	9
4 T01: an incomplete native Taylor oracle can gain false precision	9
4.1 The source-level asymmetry	9
4.2 Known zeros and unknown coefficients are different	9
4.3 A public-path regression specification	10
4.4 Why this is not the earlier hidden-analyticity finding	10
4.5 The exact finite coefficient demand	11
4.6 Logarithmic coefficients and a precise boundary rule	11
4.7 The supplied patch and its deliberately narrow scope	12
5 A01: inverse coefficient queries need a capability guard	12
5.1 A valid package object can lack an inverse model	12
5.2 Refuse before reading inverse-only fields	13
5.3 A small UX improvement with useful downstream consequences	13
6 N01: protected conditional sources obstruct automatic coverage	14
6.1 The current policy is explicit and the goal is stronger	14
6.2 A controlled conditional-polynomial witness	14
6.3 Separate a source condition from a nontransferable package contract	15

7	U01: make native diagnostic status harder to misread	15
7.1	The documented status is syntactic	15
7.2	A proposed orthogonal classification	16
8	Performance: a bounded opportunity tied to T01	16
8.1	No new native timing claim is supported	16
8.2	Benchmark what the change actually promises	17
8.3	A stronger regression property than matching one polynomial	17
9	Development recommendations with concrete acceptance gates	17
9.1	First gate: a validated regular Taylor ingress contract	17
9.2	Second gate: message-free cross-kind API refusal	18
9.3	Third gate: condition-preserving native routing	18
9.4	Fourth gate: diagnostic labels without proof inflation	18
9.5	Longer-term extension: honest lower-order observable returns	18
10	Reproduction and artifact guide	19
10.1	What actually ran	19
10.2	What remains unrun	19
10.3	Commands and file responsibilities	20
11	Conclusions and order of work	20
A	Novelty ledger and exclusions	21
B	Source inspection inventory	22
C	Independent oracle details	23
D	Limitations that should accompany any reuse	23

1 Scope, evidence, and the nonduplication boundary

1.1 A reproducible revision rather than a moving branch

All source conclusions in this article refer to 6687962f3c858a4f93623cfc496f33e35c6763d4. Its commit time is September 10, 2026, in UTC but September 9 in the user's Pacific time zone. The repository was read through the GitHub connector at that revision. A local clone was not obtained because direct network access from the container was unavailable. The article therefore does not claim a locally scanned checkout, a full static analyzer run, or a complete executable source mirror.[1]

This matters because the current package name and source organization differ from many historical reports. The public context is `AsymptoticAnalysis`; `AsymptoticExpansion`, its held alias `AsymptoticExpand`, and `AsymptoticInverse` serve different roles. The canonical modular files and the standalone entry should not be treated as two independent implementations. The inspected loader assembles the specialized modules into one package.[2, 4]

The investigation followed the public forward entry, native dispatch, ordinary jet arithmetic, the observable and composition paths, the ordinary inverse constructor and coefficient accessor, the certificate implementation, native special-function import, the Zeta/Lerch adapters, and structured refinement requests. The appendix lists the actual source windows. Family-specific Gamma, Barnes, Fourier, flat-sector, and reciprocal-log implementations were not each subjected to a complete independent line-by-line audit. Their place in the architecture is identified from the loader, public interfaces, and maintained repository accounts. This is a deep boundary-focused incremental review, not a claim of exhaustive verification of every module.

1.2 What was used to exclude earlier findings

The exclusion baseline consists of the maintained `CODE_REVIEW_STATUS.md`, both review-wave indexes, and selected original finding records relevant to the prospective deltas. The indexes describe nine reports in each wave. The first wave mostly examined 07a9781, with one report at 75de875; the second wave examined 921387e. They explicitly distinguish native observations, independent arithmetic, patch fixtures, and unrun regression programs. [10, 11, 12]

The most relevant original comparisons are review 4's native-tail and capability candidates, review 6's coefficient accessor and implicit-schema findings, and review 11's hidden-analyticity counterexample. Their finding records were inspected rather than inferred from filenames. [13, 14, 15]

T01 is related to C06/C16 but not equivalent to them. It does not depend on an unknown logarithmic degree or a nonanalytic function pretending to be analytic. Its witness is analytic and the returned *short* Taylor series is truthful. The mistake is promotion of its unknown coefficients. A01 is a concrete cross-kind refusal case under the existing D02/C11 discussion, not a proposal to invent a universal object schema. N01 sharpens the current compatibility plan's admitted incompleteness. U01 is a proposal about a newly explicit native status contract, not an allegation that the documented status has a different meaning. These distinctions are also recorded in `evidence/novelty_ledger.csv`.

The baseline comparison is comprehensive at the maintained-register level. It is not an assertion that every paragraph of all eighteen articles was reread or that an exhaustive textual search proved absolute novelty. Each claimed delta identifies its nearest known predecessor and the actual added argument. No old unresolved defect is silently relabeled as a new discovery.

1.3 Evidence classes used below

Source-established means a specific conditional, lookup, or arithmetic transition appears in the inspected revision. **Mathematically established** means an independent identity or proof validates a countermodel or a proposed precision rule. **Native-predicted** means a supplied Wolfram expression should traverse that path according to the inspected code, but the expression was not executed here. **Proposed** identifies a policy or implementation change rather than current behavior.

These categories must not be collapsed. A synthetic text fixture proves that a patch emitter inserts one guard and refuses mismatched source; it does not prove that the package loads after the insertion. A rational identity proves that an error cannot be cubic; it does not prove that a particular kernel's evaluator will reach the suspect path without a different earlier failure. The included native scripts preserve exactly that remaining verification obligation.

2 Architecture and mathematical contracts

2.1 The positive local coordinate is part of the result

The ordinary real engine normalizes source approaches using a positive small coordinate: $w = x - x_0$ from above, $w = x_0 - x$ from below, $w = 1/x$ at positive infinity, and $w = -1/x$ at negative infinity. Realness, sign, and eventual-domain checks are attached to this chart. An expression such as $w^{\sqrt{2}}$ is unambiguous on $w > 0$ even when there is no rational exponent lattice. [2], source lines 563–592

The relevant internal jet has the form

$$J(w) = \sum_{\beta \in S} w^\beta P_\beta(\log w) + \mathcal{O}\left(w^\rho \mathcal{L}(w)^d\right), \quad \mathcal{L}(w) = 1 + |\log w|, \quad (1)$$

where the finite support S is ordered and coefficients are polynomials in a logarithmic variable. The implementation stores the rows and the precision pair (ρ, d) separately. Infinity in the precision slot is a representation of exactness, not a very large finite truncation order. [2], source lines 327–392

Derived results may be factored as an exact offset plus an exact prefactor multiplying a jet. This is essential when the carrier is exponential or when an inverse construction exposes a distinguished core. An absolute error must then include the absolute value of the prefactor. The inspected `seriesMake` does this explicitly and retains an operation recipe. [3], source lines 1–150

This structure explains both the package's strengths and its obligations. A coefficient is not the same thing as an exponent block, and a block cutoff is not the same thing as a native order argument. A finite displayed expression is not the entire approximation object. A derivative may require a stronger remainder premise than the original magnitude bound. Those distinctions already exist in the package and are not proposed here as new features. [3, 16]

2.2 Ordinary inversion and specialized inverse families

The ordinary constructor reduces a local forward model to a leading monomial and small power-log corrections, schematically

$$y - y_0 = au^p \left(1 + \sum_{j=1}^m u^{\delta_j} P_j(\log u) \right). \quad (2)$$

The selected branch fixes the real interpretation of the leading uniformizer. Its correction weights are generated by sums $k_1\delta_1 + \dots + k_m\delta_m$. The source offers direct Lagrange, grouped Lagrange, and Newton routes, with frontier and precision logic shared across them. It also offers exact-termination recognition and incremental state for term goals.[2], source lines 873–1030

Logarithmic coefficient transport is not merely multiplication of scalar coefficients. For example, the core recurrence for expanding $(1 + U)^a P(\ell + \log(1 + U))$ is

$$Q_0 = P, \quad Q_{k+1} = \frac{(a - k)Q_k + Q'_k}{k + 1}. \quad (3)$$

The derivative in ℓ is necessary. The current code stops a homogeneous recurrence after a proved zero polynomial; this is an existing repair and not a new optimization claim.[2], source lines 307–322[10]

The loader additionally supplies Lambert, coordinate, core-perturbation, logarithmic, flat-sector, Fourier, Gamma, and Barnes inverse support. It would be misleading to describe these families as one unrestricted transseries field or to assume every generic operation applies to every family. For example, the ordinary series-data converter explicitly declines some Gamma/Barnes inverse operations and points to the narrower supported set. [2, 3]

2.3 Three different meanings of checking an inverse

The ordinary residual function composes a finite inverse approximation with a finite forward model and reports whether coefficients below a relative cutoff cancel. This is valuable algebraic evidence, but the returned scope is formal composition with that model. It does not by itself prove existence, uniqueness, or a numerical error bound for an arbitrary original function. [2], source lines 1173–1201

The numerical checker is a separate path. It evaluates an approximation and a reference inverse/root under working-precision and branch constraints. Its purpose is diagnostic comparison. A floating-point agreement is not a rational interval certificate.[2], source lines 1207–1223

The certificate path instead encloses an explicitly supported forward expression and its derivative on a rational interval, verifies the retained source domain, establishes a root-existence condition, and obtains a root interval and center-error bound. Its arithmetic supports a deliberately limited collection of operations, including rational arithmetic, logarithms, and exponentials. It is not a generic certificate for every special-function adapter or for every member of an unspecified remainder family. [5]

This separation is sound architecture. The present review found no additional certificate counterexample beyond the exclusions in the maintained register. That statement reports the outcome of the inspected path, not a proof of universal certificate correctness or a rerun of the recorded acceptance suite.

2.4 Native delegation is a different contract, not a shortcut proof

Explicit "Backend" → "Series" and "Backend" → "Asymptotic" select System functions before ordinary real-analytic admission. Their wrapper retains NativeResult, the held request, syntactic specifications, ambient assumptions, the producing kernel, and a native contract label. It does not assign a zero remainder simply because the native result lacks an O term. Analytic operations refuse the Native kind until an appropriate analytic contract is independently available.[4, 16]

This makes an important design distinction: broad input acceptance does not require broad analytic certification. A complex native series may be a useful and correctly preserved result even though it

cannot enter the package's real Lipschitz error calculus. Conversely, preserving a `SeriesData` object does not establish a finite logarithmic degree, derivative regularity, or uniform parameter validity for its tail. The T01 issue occurs at a different boundary: a Taylor result is actively consumed to compute a package analytic observable.

3 Comparison with built-in Wolfram Language

3.1 Compare workloads and evidence, not function names

The official interfaces make several blanket comparisons untenable. `Series` is not limited to ordinary Taylor polynomials: its documentation includes negative and fractional powers, logarithms, infinity, and successive variables. `InverseSeries` and `ComposeSeries` operate on native series objects. `AsymptoticSolve` addresses implicit equations and systems, including specified local solution branches and real-domain restrictions. These are genuine built-in alternatives, not features missing from Wolfram Language.[18, 20, 21, 23]

Workload	Built-in route	Package value and limitation
Regular local inverse	<code>Series</code> followed by <code>InverseSeries</code> ; an implicit equation can also be given to <code>AsymptoticSolve</code> .	Explicit branch/source metadata, inverse models, block access, and package error conventions can be useful. Native is a necessary baseline, not an assumed inferior method.
Fractional and logarithmic forward expansion	<code>Series</code> and <code>Asymptotic</code> already cover substantial classes.	Sparse exact real exponents and explicit logarithmic error degrees are a representation advantage when the package admits the source. Native coverage can be broader.
Several implicit unknowns or several independent variables	<code>AsymptoticSolve</code> accepts systems and multivariable specifications.	The ordinary package inverse model is scalar. General multivariable inversion is not an implemented counterpart in the inspected entry path.
Series arithmetic and composition	Native <code>SeriesData</code> , <code>ComposeSeries</code> , and ordinary arithmetic retain formal order.	Package operations carry its real chart, coefficient class, and explicit error envelope. Operation availability is scale-dependent.
Special-function asymptotics	Native <code>Series/Asymptotic</code> are broad symbolic backends.	The package combines dedicated identities/adapters with native imports and selected direct defining-sum methods. Native delegation is explicitly retained rather than replaced.
Large real Zeta or Lerch parameters	Native should be tested on the precise requested form and kernel.	The inspected direct adapters provide exact defining-sum/moment constructions and explicit tail-bound metadata under fixed-parameter conditions. No universal native failure claim is made.

Workload	Built-in route	Package value and limitation
Implicit branch selection	<code>AsymptoticSolve</code> can specify solution centers and a real domain.	Package inverse constructors and callable inverse syntax add their own branch and source-domain records; this is a different workflow, not a substitute for every native system solver.
Pointwise rigorous error	A separate verified numerical argument is required; a native truncated expression is not itself such a certificate.	<code>InverseCertificate</code> supplies one limited rational-interval route for supported explicit expressions. Its existence is not a blanket certificate for all package outputs.
Formal complex or symbolic native result	Use the native head directly.	Explicit native wrapper modes preserve the result and provenance but intentionally decline ordinary real analytic arithmetic.

Sources for the matrix are the official native references and the inspected package contracts and entry paths.[18, 20, 21, 22, 23, 19, 2, 3, 6, 4, 16, 5]

3.2 An example where a fair comparison is straightforward

For $f(x) = x + x^2$ and the inverse approaching zero from the positive side,

$$g(y) = \frac{\sqrt{1+4y}-1}{2} = y - y^2 + 2y^3 - 5y^4 + 14y^5 + \mathcal{O}(y^6). \quad (4)$$

The formula and coefficients follow by expanding the exact algebraic inverse. A comparison can use the following calls, while matching the achieved error rather than assuming that their numerical order arguments are identical:

```
AsymptoticInverse[x + x^2, {x, 0}, {y, 6}]
InverseSeries[Series[x + x^2, {x, 0, 6}], y]
AsymptoticSolve[x + x^2 == y, {x, 0}, {y, 0, 5}, Reals]
```

These calls are supplied as native comparison specifications, not transcripts of executions performed in this audit. The native observation script includes nearby versions of this control. The equality of the mathematical target does not establish equal branch metadata, output order, cost, or certificate strength.

3.3 Where the package is genuinely differentiated

The sparse power-log representation is valuable when several exact real weights generate a support that does not fit a convenient rational lattice. A model-based inverse interface can also expose the source of a coefficient and permit controlled refinement without reconstructing an entire native symbolic problem. Exact carriers, observable powers, and specialized real inverse coordinates provide useful organization for research calculations. These statements concern the inspected representation and interfaces, not an empirical assertion that the package is faster on every such example. [2, 3, 9]

The direct Zeta adapter illustrates another specific distinction. In the positive large argument S , it uses the convergent defining sum and coordinate $w = e^{-S}$, whose weights are $\log n$. Its retained

expression is a finite sum of n^{-S} ; the omitted tail has a first-term/integral-comparison bound. The Lerch adapter uses geometric moments, including finite termination for some parameters, and records that parameters are fixed during the approach. These are concrete source features. The archive’s existing questions about transporting their quantitative metadata through later operations are not reopened as new findings here.[6, 10]

3.4 Native capability is not the same as native proof strength

Asymptotic documents an asymptotic scale and an error smaller than the last retained scale member. It can return finite or infinite expressions. `AsymptoticLess` provides a way to ask a symbolic asymptotic comparison, but an unresolved comparison remains unresolved. Neither fact licenses an adapter to fill unknown Taylor coefficients with zero.[22, 24]

Similarly, `Series` treats unrecognized functions as analytic by default, with an option controlling that premise. `FunctionAnalytic` can investigate analyticity in a specified domain, but it is not a substitute for checking the actual order of an already returned series. The regularity and information questions are logically independent.[18, 25]

4 T01: an incomplete native Taylor oracle can gain false precision

4.1 The source-level asymmetry

The generic unary branch of `seriesJetApply` first computes an input jet, checks that its argument approaches a finite constant, and chooses a native Taylor working order from the requested cutoff and the valuation of the small part. It then calls `Series` at a signed local increment. [3], source lines 250–292

The subsequent admission test requires a `SeriesData` result, nonnegative starting index, denominator one, and coefficients free of the temporary variable. It does *not* check the returned endpoint. The coefficient function returns zero when an index is absent from the coefficient list. The result is passed to `pUnitSeries`, whose precision is based on the input precision and requested cutoff, not on the native endpoint. [3]

In contrast, the ordinary forward `fwdAnalytic` path explicitly checks that the returned native endpoint exceeds its requested Taylor order before using the result as an ordinary coefficient oracle. Otherwise it falls back to a native-series import path that tracks native precision. This concrete asymmetry is why a fix confined to one native importer cannot be assumed to cover the observable path.[2], source lines 477–513

4.2 Known zeros and unknown coefficients are different

A regular native series with endpoint q gives coefficients below q . A coefficient list may omit trailing *known zeros* within that interval. That is harmless. But the endpoint also marks a region where coefficients are *unknown*. The same array-out-of-range behavior must not represent both situations. Native series structures explicitly carry the omitted order, and operations on them are intended to respect that order.[19]

For a truthful, analytic countermodel, let

$$H(t) = \frac{1}{1-t}, \quad |t| < 1. \quad (5)$$

Suppose a limited symbolic oracle returns

$$H(t) = 1 + t + \mathcal{O}(t^2) \quad (6)$$

when asked for more terms. Every returned coefficient and the returned remainder in (6) is correct. A library function with its own `Series` method is allowed to know less than a caller requests; the caller must either accept the weaker precision or decline the request.

Apply this observable to the exact inner jet $t = x$ and request an exclusive cutoff of 3. With the inspected zero-filling rule, the missing quadratic coefficient is interpreted as zero while the result precision is still computed at cutoff 3. The resulting mathematical interpretation is

$$H(x) \stackrel{\text{invalid}}{=} 1 + x + \mathcal{O}(x^3). \quad (7)$$

The exact error is

$$\frac{1}{1-x} - (1+x) = \frac{x^2}{1-x}, \quad \frac{|H(x) - (1+x)|}{x^3} = \frac{1}{x(1-x)} \rightarrow \infty \quad (x \rightarrow 0^+). \quad (8)$$

Thus (7) is false. No floating-point experiment or hidden nonanalyticity is needed to prove the failure of the information transfer.

4.3 A public-path regression specification

The test fixture deliberately keeps the symbolic function inert, supplies a correct numerical meaning and value at zero, and gives it a custom, truthful, but short native Taylor method. Defining the symbolic function directly as a rational expression would allow ordinary evaluation to erase the test boundary.

```
ClearAll[shortTaylor];
shortTaylor[0] = 1;
shortTaylor[z_?NumericQ] := 1/(1-z);
shortTaylor /: Series[shortTaylor[arg_],
  {v_Symbol, 0, ord_Integer}, opts : OptionsPattern[Series]] :=
  Series[1/(1-arg), {v, 0, Min[ord, 1]}, opts];

s = AsymptoticExpansion[x, {x, 0, 4}, "Backend" -> "Package"];
SeriesObservable[s, shortTaylor[z], z, "Cutoff" -> 3]
```

The source-predicted problem is the production of $1 + x$ with cubic package precision. This expression was not executed in a native kernel here. The fixture's own direct `Series` result is therefore tested first in the supplied `.wlt` file; a failure of that control would invalidate this particular native reproduction without invalidating the information-theoretic countermodel or the missing source check.

An additional natural reachability probe is a changed native `SeriesTermGoal` default that causes a requested native order to return a shorter series. The observable call does not explicitly reset that option. Whether a specific built-in and kernel actually return a shorter endpoint under that configuration must be measured; this report does not assert that they do. The custom oracle is the controlled reproduction specification, not a claim that default `Sin`, `Cos`, or every special function currently fail.

4.4 Why this is not the earlier hidden-analyticity finding

Review 11's N03, now represented by C16 in the register, used an opaque real function with truthful low derivatives but an unjustified higher-order analytic remainder. Here the source is genuinely

analytic, its complete Taylor series is known, and even the limited oracle's remainder is true. The new issue persists after granting all the regularity that the earlier example lacked. Likewise, the example has no logarithms, so correcting a missing logarithmic tail degree cannot repair it.[15, 10]

The general principle is that stronger source regularity does not reveal an unknown coefficient's value. Analyticity says that a Taylor expansion exists; it does not say that every coefficient not supplied by an oracle is zero. T01 is therefore an additional precision-consumption obligation, not another request to add an analyticity check.

4.5 The exact finite coefficient demand

Proposition 4.1 (Required endpoint for a regular observable). *Let $w \rightarrow 0^+$ and let the small retained input U have valuation $\alpha > 0$. Suppose the effective positive output cutoff is c and coefficient powers are retained only for exponents strictly below c . Put*

$$N = \left\lceil \frac{c}{\alpha} \right\rceil. \quad (9)$$

For a generic regular Taylor observable, coefficients of U^k can be needed only for $0 \leq k < N$. A native regular Taylor result with endpoint $q \geq N$ contains all these coefficients. If $q < N$, zero-filling the unknown coefficients is unsound in general.

Proof. Every term of U has exponent at least α , so every term of U^k has exponent at least $k\alpha$. If $k \geq N$, then $k\alpha \geq c$ and no such term is retained. Conversely, take $U = w^\alpha$ and $H(t) = 1/(1-t)$. Its coefficient of U^q is one. When $q < N$, $q\alpha < c$, so that coefficient belongs below the requested cutoff. An oracle stopping at q has not supplied it. Replacing it with zero omits a genuinely required term. $\square$

This criterion uses the *returned* endpoint, not the requested order and not merely the length of the stored coefficient list. Endpoint equality is sufficient for this regular, exclusive-cutoff branch. Requiring $q > N$ is a safe but unnecessarily strict guard here; the separate core path also handles singular native forms and should not be mechanically rewritten using the regular-only argument.

For an input remainder $e = \mathcal{O}(w^P \mathcal{L}(w)^D)$, the effective cutoff in the existing unit-series routine is $c = \min(h, P)$, where h is the requested cutoff. Under the regular observable premise, bounded local derivatives give

$$H(c_0 + U + e) - H(c_0 + U) = \mathcal{O}(e). \quad (10)$$

This is separate from the Taylor-tail contribution. The proof requires a regular observable near c_0 ; it must not be reused for a pole or a branch point without its own transport rule.

4.6 Logarithmic coefficients and a precise boundary rule

Suppose $U = \mathcal{O}(w^\alpha \mathcal{L}(w)^d)$ and the native regular Taylor tail is $\mathcal{O}(t^q)$. Its composition contributes

$$\mathcal{O}\left(w^{\alpha q} \mathcal{L}(w)^{dq}\right). \quad (11)$$

If $\alpha q > c$, any fixed logarithmic power is absorbed by the strict power margin. If $\alpha q = c$, the boundary logarithmic degree must include dq . In the current unit-series precision formula, N times the maximum retained input logarithmic degree is already used at that boundary; when $q = N$, it covers (11). The supplied guard therefore preserves the existing nonlinear boundary accounting instead of replacing it. [2], source lines 374–393

For an implementation that chooses to return a weaker result instead of a failure, a basic candidate exponent is

$$P_{\text{out}} = \min\{h, P, \alpha q\}. \quad (12)$$

The logarithmic degree at an active boundary must include each contribution attaining that exponent, together with actually discarded finite products. The short patch does not implement this more involved weakening policy. It refuses $q < N$ before the coefficient oracle can silently promote information.

4.7 The supplied patch and its deliberately narrow scope

The emitter inserts the following checks after the existing regularity guard and before coefficient extraction:

```
If[native[[1]] != u || native[[2]] != 0,
  fail["UnsupportedObservableNativeCoordinate", ...]];
If[! IntegerQ[native[[4]]] || ! IntegerQ[native[[5]]] ||
  ! TrueQ[native[[5]] >= n],
  fail["InsufficientObservableNativeOrder", ...,
    <|"RequiredTaylorEndpoint" -> n,
      "ReturnedTaylorEndpoint" -> native[[5]],
      "NativeResult" -> native|>]];
```

Here n is the already computed N in (9). The existing check has established denominator one and the absence of a negative starting index. The new chart check prevents accidental use of an unrelated native coordinate. It is defensive validation, not an additional independently observed bug.

`observable_order_patch.py` emits a unified diff by default and never modifies the source file. A candidate copy can be written only to a distinct, nonexistent path. The emitter requires exact adjacent source sentinels and refuses a changed, duplicated, displaced, or already-patched insertion point. Those safeguards were tested on a synthetic fixture. They are not a native syntax check or integration test.

The patch does not change the hidden-analyticity policy, log-tail import, coefficient realness, generic resource limits, or nonlinear frontier algorithm. It does not handle singular observable Taylor forms. Its focused obligation is only this: no coefficient that the native endpoint leaves unknown may be used as a known zero below the observable's required cutoff.

5 A01: inverse coefficient queries need a capability guard

5.1 A valid package object can lack an inverse model

The object overload of `InverseExpansionCoefficient` has explicit refusals for Native results and logarithmic scales. Otherwise it builds an association from a `["Model"]` and passes it to the ordinary model overload, also reading the stored inverse power. The ordinary forward constructor produces a valid `GeneralizedSeries` object with rows, precision, source, and expression, but without an inverse `Model` or inverse `Power`. [2], source lines 689–724 and 1250–1267

A minimal characterization request is therefore:

```
s = AsymptoticExpansion[1 + x, {x, 0, 2}, "Backend" -> "Package"];
InverseExpansionCoefficient[s, {0}]
```

There is no meaningful ordinary inverse coefficient model in `s`. Refusal is appropriate. The issue is that the object overload can read absent inverse-only fields and construct a `Join` with a `Missing` value before declining. The source predicts an avoidable message or a less specific fallback failure. The exact native message sequence and final output were not observed here and are left to the supplied characterization script.

This is not the already recorded C09 problem in which a stored power shadows an explicit option. It is also not a malicious or hand-forged association: the object is produced by a normal public forward constructor. It gives the existing capability discussion a concrete failure boundary and a small test that does not require redesigning all object schemas.[14, 10]

5.2 Refuse before reading inverse-only fields

A minimal capability check asks whether a result contains an association-valued model with the ordinary coefficient routine's required keys:

```
model = Lookup[a, "Model", Missing["NoCoefficientModel"]];
required = {"Gaps", "Polynomials", "LeadingPower",
            "LogVariable", "Symbolic"};
If[! AssociationQ[model] ||
    ! AllTrue[required, KeyExistsQ[model, #] &],
    Return[Failure["UnsupportedCoefficientModel", ...]]];
```

A result's `Kind` alone should not be used as a universal capability predicate: some adapted inverse results may carry a usable ordinary model, while other objects with inverse-related names do not. Check the contract needed by the particular operation, then delegate. A broader import/schema validator remains the separate issue already recorded in C11/D03.

The supplied `CheckedInverseCoefficient` helper implements this narrow preflight without altering upstream definitions. It deliberately preserves existing delegation and does not claim to fix option precedence. Its desired behavior on a forward object is a message-free, specific `Failure`. Its ordinary-inverse control compares its output with the current underlying function. Both are unrun Wolfram specifications.

5.3 A small UX improvement with useful downstream consequences

This boundary matters for generic notebook tooling: a user can legitimately hold a heterogeneous list of `GeneralizedSeries` results and ask what operations are available. A low-level association or `Join` message is a poor explanation that the selected operation needs an inverse model. The failure should name the requested capability and actual result kind, and leave the original object untouched.

It would be counterproductive to manufacture an inverse model for a forward object solely to make this accessor accept it. Forward coefficient extraction and inverse multi-index coefficient extraction are different questions. The right fix is a precise refusal or an explicitly different forward-coefficient operation, not implicit reinterpretation of the object.

6 N01: protected conditional sources obstruct automatic coverage

6.1 The current policy is explicit and the goal is stronger

The compatibility plan says that completely subsuming successful System‘Series and System‘Asymptotic inputs is an accepted objective. It also states that automatic coverage is not yet complete. In particular, source syntax containing ConditionalExpression is protected and retained on the package path. This is intentional current behavior, not an undocumented accident.[17, 4]

The additional point is structural: improving the fallback failure list or adding a second native-backend probe cannot close coverage gaps that occur *before either native backend is allowed to run*. The protection predicate is itself part of the coverage boundary.

6.2 A controlled conditional-polynomial witness

Consider the complex polynomial $1 + ix$ with a parameter condition $a > 0$:

```
Series[ConditionalExpression[1 + I*x, a > 0],
  {x, 0, 2}, Assumptions -> a > 0]

AsymptoticExpansion[ConditionalExpression[1 + I*x, a > 0],
  {x, 0, 2}, Assumptions -> a > 0, "Backend" -> "Series"]

AsymptoticExpansion[ConditionalExpression[1 + I*x, a > 0],
  {x, 0, 2}, Assumptions -> a > 0]
```

The first two calls are the direct-native and explicit-wrapper controls. A native result may retain the condition or discharge it under the assumptions; either is acceptable provided the result is equivalent to $1 + ix$ under $a > 0$. The third call is protected by its original conditional syntax. If it reaches ordinary real admission, the explicit complex coefficient cannot be accepted as a real analytic coefficient. There is no mathematical difficulty in the native polynomial expansion; the obstruction is representation routing. [4, 2, 16]

These are native reproduction specifications, not executed outputs. The characterizer records all three results and an unconditional automatic control. It does not assume a particular pretty-printed conditional form or turn a native unresolved call into a passing result. The expected control relationship is semantic equality under $a > 0$, together with an actual computed native representation.

The example uses no Direction option. The current official references for Series and Asymptotic do not list that option; it is therefore not used to claim their input coverage. Direction support in a different function, such as AsymptoticSolve, cannot be transferred to these two interfaces by analogy. This is an important constraint on the comparison harness.[18, 22, 23]

Proposition 6.1 (A routing obstruction independent of backend search). *Suppose a dispatcher irrevocably selects a real-only engine whenever a source has a syntactic condition. Suppose there is a conditioned complex polynomial that a native backend computes. Then that dispatcher cannot, under that policy, be an automatic input superset of the native backend, regardless of what second-backend search is implemented after an unprotected native attempt.*

Proof. The conditioned source enters the protected real-only path. Its complex coefficient prevents admission to that representation. By hypothesis the protection is irrevocable, so neither native backend

is reached. Changes to a search that runs only after a native attempt do not affect this execution path. □

The proposition is a control-flow fact. The included native controls are needed to establish the concrete witness on a producing kernel. This is a sharper coverage obligation than merely observing that the dispatcher probes one native backend, but it does not contradict the repository's candid statement that complete automatic coverage remains open.

6.3 Separate a source condition from a nontransferable package contract

A condition is not automatically a request for a real analytic remainder. A suitable compatibility change would distinguish at least two cases. An explicit package branch selection or resource requirement cannot simply be ignored by a native engine that has no corresponding semantics. A source `ConditionalExpression`, however, can sometimes be preserved verbatim in a native request and in the native result without asserting a package branch proof. The latter case merits a narrow routing rule rather than blanket protection solely because a condition is present.

The safe migration is not to strip conditions, suppress realness failures, or reclassify a native complex result as real. It is to permit a clearly labeled native result when the whole source condition and applicable native assumptions are preserved. A first implementation can restrict the rule to ordinary conditional expressions with no package-only branch/resource options and no embedded package series objects. Cases outside that rule should retain an explicit refusal.

Any such rule must also preserve the source-evaluation policy. The existing contract already warns that a package attempt may materialize a source and common option values before native fallback. A compatibility change should not promise arbitrary side-effect equivalence with a direct native call. Tests should record the held request, source evaluation count, condition, selected head, and native result separately. This recommendation specializes the current documented policy; it is not a proposal to replay arbitrary user programs repeatedly until one happens to succeed.[16]

7 U01: make native diagnostic status harder to misread

7.1 The documented status is syntactic

`NativeEvaluationStatus->"Computed"` is currently defined as the absence of remaining `Series` or `Asymptotic` calls. This definition is stated in the result-contract document and implemented by a `FreeQ` test. It is not a claim of a successful mathematical proof or even the absence of a failure marker.[4, 16]

Consequently, a returned `$Failed`, a `Failure` object, or a container with such a value can satisfy the syntactic predicate. The code is consistent with its own definition; this is not reported as a correctness defect. The UX risk is that generic consumers may read “Computed” as “successful” and color an otherwise failed native result as successful.

The raw result must remain available whatever diagnostic classification is added. A failure marker can legitimately be a user's input constant, and an indeterminate expression need not indicate an implementation defect. The classification is a presentation/automation aid, not a proof of the meaning of every returned expression.

7.2 A proposed orthogonal classification

A small, nonbreaking addition could distinguish a structural property, `NativeCallResolvedQ`, from an outcome label:

Proposed label	Interpretation
Failed	The whole result is a standard failure marker.
ContainsFailure	A nested result contains such a marker; preserve partial output.
Unresolved	A native expansion call remains unevaluated.
Indeterminate	The result contains <code>Indeterminate</code> ; do not equate this with an unresolved native call.
ReturnedWithoutFailure Marker	None of the preceding syntactic markers was found. No correctness, exactness, or analytic-validity proof is implied.

The helper `NativeOutcome` implements this proposed classification. It does not replace the current documented field. Conditional results and zero are valid returned expressions, not failure markers. Divergence is not classified as a failure merely because `Infinity` occurs. These controls prevent a UI improvement from becoming an overzealous mathematical filter.

8 Performance: a bounded opportunity tied to T01

8.1 No new native timing claim is supported

The repository already has a substantial performance backlog. Recommending binary powering, sparse export, factorization budgets, cache profiling, or method-specific enumeration again would not meet the user's nonduplication requirement. The present contribution is narrower: derive the exact finite Taylor coefficient demand at one consuming boundary and use that demand to avoid both insufficient information and unnecessary coefficient requests. [10, 14]

The inspected observable path requests native Taylor order $N = \lceil c/\alpha \rceil$. Proposition 4.1 shows that, in the regular branch, coefficients through degree $N - 1$ suffice for an exclusive cutoff. If a native request through $N - 1$ reliably returns endpoint at least N , one could request that order instead. This removes one unnecessary requested Taylor degree. It does not prove that the underlying symbolic engine performs one less derivative, nor that total time falls by a particular percentage.

c	α	Current requested order N	Minimal generic degree needed
3	1	3	2
5	2	3	2
4	2	2	1
10	1/2	20	19
10	1/8	80	79

These are exact operation-demand counts, not wall-clock benchmarks. The benefit may be negligible for elementary functions and important near an expensive symbolic coefficient or a resource boundary. Measurement is required before changing the request order. The supplied patch leaves the current request order unchanged and only validates the returned information.

8.2 Benchmark what the change actually promises

A useful focused experiment compares unmodified source, the guard-only patch, and a separate minimal-order variant. Use identical regular observables and inner jets, the same assumptions, and the same achieved package error. Record the requested native degree, returned native endpoint, finite coefficients, source messages, failure category, elapsed time, and expression size. A weaker returned error is not a faster solution of the same problem.

Controls should include monomial inputs of several positive rational valuations; logarithmic coefficients with a boundary product exactly at the cutoff; a native series whose trailing stored coefficients vanish; and a truthful oracle that refuses to increase its endpoint. The latter should fail promptly or return honest weaker precision, not loop indefinitely or report a stronger error. The provided short-oracle and scaled-valuation tests are the first members of this experiment, not a measured benchmark suite.

A source-level asymmetry worth preserving in the benchmark is that the forward analytic path and the observable path have different admissible native shapes. The forward path handles some Laurent/Puiseux forms; the ordinary observable path rejects them. Optimizing both by replacing them with a single unqualified Taylor helper would risk changing branch and remainder semantics. Share the information check where its hypotheses match, not every surrounding behavior. [2, 3]

8.3 A stronger regression property than matching one polynomial

The key performance/correctness invariant is information monotonicity. When the native oracle returns less coefficient information, the consumer may fail, return a weaker valid object, or make a bounded new request. It must not return the same strong precision with newly invented zero coefficients. Conversely, a complete oracle and a shorter-but-sufficient oracle should produce equivalent retained coefficients at the requested cutoff.

This property is inexpensive to test without a large special-function catalog. For each positive α , cutoff c , and endpoint q , a geometric oracle with all coefficients equal to one has an exact first unknown term at $q\alpha$. It distinguishes insufficient information from cancellation and from stored trailing zeros. The Python model enumerates this boundary using exact rational arithmetic, so no threshold decision depends on floating-point rounding.

9 Development recommendations with concrete acceptance gates

9.1 First gate: a validated regular Taylor ingress contract

The next useful abstraction is not a global registry rewrite. It is a small record at this particular boundary containing the native chart, coefficient range, returned endpoint, coefficient dependence checks, and the regularity premise under which the series is being consumed. A consumer must ask whether that range covers its actual finite demand before iterating coefficients. This directly implements the missing T01 check and gives the existing D02 capability proposal a narrow, testable starting point.[10]

A robust coefficient accessor for that record should distinguish `KnownCoefficient[value]`, `KnownZero`, and `UnknownCoefficient`. An array position outside the stored list but below the native endpoint is a known zero only according to the native representation's normalization. A position at or above the endpoint is unknown. The consumer can then refuse an unknown coefficient instead of relying on a

global implicit zero-fill convention.

Acceptance should require the exact short-oracle witness to be refused or weakened, complete-oracle controls to remain unchanged, equality of the returned endpoint with the minimal demand to remain accepted, and logarithmic boundary products to retain their degree. It should also record the actual number of native oracle calls so a validation change does not silently introduce unbounded retries.

9.2 Second gate: message-free cross-kind API refusal

Add the inverse coefficient model check before accessing inverse-only keys. The concrete acceptance criterion is not simply `FailureQ`: a generic fallback failure after low-level messages would still meet that weak test. Require the intended failure tag, no unexpected messages, and no change to an ordinary inverse control. Retain the separate C09 option-precedence tests; passing the new kind-boundary test says nothing about that older issue.

This style of cross-kind test can be extended to other operations using valid publicly constructed objects. It is sharper than fuzzing arbitrary malformed associations because it tests an actual interaction users can reach without violating object-construction conventions. The broad capability/schema problem is already known; the new work is a targeted witness and a refusal contract.

9.3 Third gate: condition-preserving native routing

The routing acceptance test should pair an unconditioned native-compatible expression with the same expression under an admissible parameter condition. The condition may change the validity domain but should not accidentally force an otherwise representable native result into a real-only representation. Every passing case must retain the original condition or show its discharge under recorded assumptions.

Do not generalize this rule to package-only `InverseFunctionBranches`, explicit package resource limits, or arbitrary existing package remainders. Those can impose requirements that native heads do not express. A request that cannot transfer its requirements should return a precise conflict rather than silently weaken them. The policy change and its tests should be developed separately from T01, because they affect coverage and evaluation semantics, not the truth of the ordinary Taylor precision formula.

9.4 Fourth gate: diagnostic labels without proof inflation

Add any proposed native outcome label as an orthogonal field. Maintain the raw native output and its formal/asymptotic contract. Require tests for a zero result, a conditional result, a whole failure marker, a nested failure marker, and an unresolved native call. Do not mark a result analytically certified because no failure marker was found. This is a small UX improvement, not an opportunity to merge formal and analytic result kinds.

9.5 Longer-term extension: honest lower-order observable returns

Once the guard-only change is natively validated, a useful extension is a selectable policy for an oracle that returns too little information: `"InsufficientOrderAction"` -> `"Fail"` or an explicitly documented `"ReturnWeaker"`. The latter would use the transported native endpoint, retain its evidence, and state

the achieved precision separately from the requested precision. This is a specific realization of the known request/result contract problem, not a new general roadmap item.

The extension should not fabricate a higher-order Taylor series by repeatedly asking the same capped oracle. A bounded retry policy can request an improved endpoint, but improvement must be established from the returned data. A finite native series that is unchanged after a larger request supplies no additional information. Existing core native-tail reconciliation already follows this principle in a different path; the proposed work is to apply an analogous information discipline to regular observables.[2], source lines 542–563

The broad proposals already listed in the archive—complex sectors, multirate transseries, uniform parameter asymptotics, multivariate systems, and other asymptotic operator adapters—are not repeated here. These boundary changes are useful prerequisites because every larger extension introduces more places where one representation consumes another representation’s precision.

10 Reproduction and artifact guide

10.1 What actually ran

The Python checker uses only standard-library exact integers and fractions. It does not load the repository or emulate Wolfram evaluation. Its populations are deliberately separate:

Population	Passed checks
Exact rational remainder identities	4
Exact rational lower inequalities for the false cubic bound	4
Rational finite-demand and endpoint-equivalence cases	868
Sparse rational-power geometric polynomial identities	56
Synthetic text-patch fixture checks	8
Total	940

There were zero failing assertions in these populations. They ran under Python 3.13.5. The full case list is in `evidence/independent_results.json`. These counts must not be added to historical native acceptance totals or described as upstream tests.

For $x = 10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}$, the exact ratios $|H(x) - (1 + x)|/x^3$ recorded by the oracle are

$$\frac{100}{9}, \frac{10000}{99}, \frac{1000000}{999}, \frac{100000000}{9999}. \quad (13)$$

The divergent limit was proved in (8); these rational values are reproducible illustrations, not a substitute for that proof.

10.2 What remains unrun

No successful native Wolfram invocation was available. The service failed on the context request and on version/evaluation probes. The supplied `TaylorIngressRegressions.wlt`, `BoundaryPolicyRegressions.wlt`, `ReviewAdapters.wl`, `characterize.wl`, and the emitted patch have not been natively syntax-checked, loaded, or integrated into the full upstream test suite. This limitation is represented explicitly in `evidence/execution_scope.json`.

The Wolfram fixtures should be run in a fresh kernel. First verify the direct short-oracle control. Then characterize the baseline before staging any patch. Apply only the T01 source change for its focused acceptance run. A01's helper and N01's proposed routing policy are separate artifacts and must not be reported as fixed by a T01-only patch. Some desired-contract tests are expected to fail on the pinned source; a failing proposed-policy test is not evidence that the current documented policy was violated.

10.3 Commands and file responsibilities

From the archive root, rerun the independent checks with:

```
python code/independent_checks.py --output evidence/independent_results.json
```

Produce a patch against a local canonical source file without modifying it:

```
python code/observable_order_patch.py \
  /checkout/Asymptotic/src/Kernel/SeriesOperations.wl > observable-order.patch
```

The emitter refuses an unexpected source shape. Review the diff before applying it to a scratch checkout. When using the standalone distribution, regenerate it with the repository's documented build process after changing canonical source; do not patch only one representation and then test a different unchanged entry.

Record native baseline observations with:

```
wolframscript -file code/characterize.wl \
  /checkout/Asymptotic/AsymptoticAnalysis.wl native-observations.json
```

The script records the actual kernel version, runtime native option lists, messages, timeouts, result strings, and the selected package path. It records but does not automatically verify the declared repository revision. It makes no repository modifications and does not treat every returned expression as a passing test.

After loading the intended package in a fresh kernel, the focused suite is:

```
TestReport["code/TaylorIngressRegressions.wlt"]
```

The convenience `run_focused.wl` selects exactly that nonempty file and prints the report. It is intentionally not a release gate and does not assert an exit-code acceptance result. The policy/helper suite is run separately after `Get["code/ReviewAdapters.wl"]`. Inspect all individual test outcomes.

The article can be rebuilt using the included shell script or three ordinary `pdflatex` passes from the article directory. The delivered PDF was built and visually checked locally; its production is independent of native package execution.

11 Conclusions and order of work

The strongest additional technical issue is T01: a consumer of native Taylor coefficients must respect the native endpoint. The omission is visible in the observable path, a neighboring forward path already performs an order check, and the exact geometric countermodel shows why the missing check matters. The supplied guard is small enough to validate independently and does not require a redesign of the package's asymptotic calculus.

The next improvements are contract boundaries rather than new symbolic algorithms. A01 should reject a valid but unsuitable object before accessing inverse-only metadata. N01 requires an explicit decision about conditions that can be preserved by native delegation, because a protected branch cannot be repaired by a backend search that never runs. U01 would make native status less misleading while retaining the distinction between a returned expression and a proved analytic result.

Order	Change	Required evidence
1	Stage T01's chart/endpoint guard; keep native request order unchanged.	Native short-oracle failure and complete/boundary controls.
2	Add the inverse coefficient capability check.	Specific message-free refusal and unchanged inverse control.
3	Decide and implement the narrow conditional native route.	Direct, explicit, and automatic native comparison with retained conditions.
4	Add orthogonal native outcome diagnostics.	Failure, partial, conditional, zero, and unresolved controls.
5	Benchmark a separate minimal Taylor-request variant or honest weaker-return policy.	Identical achieved precision, recorded native endpoints, and measured cost.

A fair overall assessment is therefore neither “the built-ins make this package redundant” nor “the package subsumes Wolfram Language.” Its specialized real asymptotic representation and inverse workflow have clear value, while native symbolic coverage remains broader in important directions. The current explicit native boundary acknowledges that fact. The highest-value incremental development is to make every transition between these representations preserve exactly the information and validity it actually possesses.

A Novelty ledger and exclusions

Item	Nearest predecessor	What is added here
T01	C06, C16; review 4 R03; review 11 N03	A truthful analytic short Taylor oracle, a different consuming path, a missing endpoint check, an exact demand theorem, and a narrowly scoped guard. No claim that unknown-function analyticity or all native-tail issues are newly discovered.
A01	D02/C11; review 6 A11; nearby C09	A public forward object entering inverse-only metadata access; a message-free capability refusal. Stored-power precedence is deliberately not re-reported or fixed.

Item	Nearest predecessor	What is added here
N01	Current native compatibility plan	A conditioned complex polynomial and a control-flow proof that post-native backend search cannot overcome pre-native protection. Current documented incompleteness is acknowledged.
U01	Current native result contract	A proposed distinction between call elimination and failure markers. The current Computed definition is not misrepresented as a success guarantee.
Excluded	C01–C07, C08–C23 where already recorded	No recycled proofs or repair recommendations for the archived correctness backlog. C06/C16 appear only to delimit T01.
Excluded	P01–P08	No recycled resource, enumeration, factorization, or cache roadmap. Only the observable’s proved finite Taylor demand is developed.
Excluded	D01–D10, V01–V02, X01–X12	No repeated general feature wishlist, release-gate proposal, or broad schema rewrite. The report supplies narrowly attributed realizations where a new witness is present.

The register and wave indexes are the authoritative baseline for these references. The ledger does not claim that the excluded items are all fixed; it means they were not counted again as new work in this article. [10, 11, 12, 13, 14, 15]

B Source inspection inventory

The following windows identify the actual basis of the source arguments. A listed file was not necessarily read in one untruncated response. Overlapping reads were used for the important transitions. Family-specific modules outside this table were not each independently audited in full.

Canonical source	Windows	Inspected role
AsymptoticAnalysis.wl	Selected 1–185, 290–753, 870–1030, 1170–end	Public contracts, assumptions, jet arithmetic, native Taylor ingress, forward object construction, inverse construction, coefficient access, loader.
SeriesOperations.wl	Selected windows across file; focus 248–290	Data representation, observable Taylor consumer, composition, derivative and refinement dispatch; T01 location.
NativeCompatibility.wl	1–end	Held argument preparation, protected source classifier, native routing, result metadata and status.

Canonical source	Windows	Inspected role
InverseCertificates.wl	1–end	Rational enclosures, domain proof, root/center bounds, accuracy and refinement controller.
NativeSpecialFunctions.wl	1–end	Native series-tree import, carrier/error transport, real-domain and achieved-error checks.
DirichletSpecialFunctions.wl	1–end	Direct Zeta/Lerch construction, parameter scope and explicit remainder metadata.
InverseFunctionExpressions.wl	Selected 1–245	Forward dispatch, callable/inverse handling and conditional-input proof path.
RefinementRequests.wl	1–end	Separation of additional-block requests from numerical certificate requests.

Documentation read includes the full review-wave indexes, the maintained finding register in multiple windows, `NATIVE_RESULT_CONTRACTS.md`, the relevant compatibility-plan sections, and the selected original ledgers listed in the bibliography. Official Wolfram reference pages were checked during this review. Runtime behavior beyond the source remains subject to the native validation limits already stated.

C Independent oracle details

For a rational valuation α , the finite geometric polynomial $G_q(w) = \sum_{k=0}^{q-1} w^{k\alpha}$ satisfies the exact identity

$$(1 - w^\alpha)G_q(w) = 1 - w^{q\alpha}. \quad (14)$$

The sparse oracle represents exponents by exact Python fractions and performs this multiplication independently of the package’s jet functions. Thus the first omitted term is known algebraically rather than inferred from a sampled numerical difference. The demand checks enumerate positive rational valuations and cutoffs, prove by exact comparison that $q \geq \lceil c/\alpha \rceil$ is equivalent to $q\alpha \geq c$, and verify that q is a required coefficient index whenever that inequality fails.

The synthetic patch tests verify one insertion, preservation of the existing regularity test, the nonstrict endpoint comparison appropriate to an exclusive cutoff, and rejection of absent, duplicate, displaced, or already-patched source sentinels. They deliberately operate on a small fixture rather than pretending the unavailable full repository was loaded in the container.

D Limitations that should accompany any reuse

The proposed patch is not a reviewed upstream commit and has not passed native integration tests. The test fixtures themselves still require native validation. The mathematical countermodel establishes a precision obligation, not the frequency of the defect in default built-in workloads. The API message sequence in A01 and the exact native conditional output in N01 are predictions. U01 is a policy proposal consistent with, rather than a refutation of, the documented current status definition. No benchmark establishes a native speedup.

These limits do not make the source arguments speculative in the same way. The missing observable endpoint comparison, the inverse-model lookup on the object path, and the conditional-source protection are present in the inspected revision. The archive preserves the source pointers, proof, executable model, and native observation specifications needed to turn each remaining empirical question into a bounded check.

References

- [1] Vladimir Reshetnikov, *Asymptotic*, commit 6687962f3c858a4f93623cfc496f33e35c6763d4. <https://github.com/VladimirReshetnikov/Asymptotic/commit/6687962f3c858a4f93623cfc496f33e35c6763d4>.
- [2] *AsymptoticAnalysis.wl*, canonical core at the pinned revision. <https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/src/Kernel/AsymptoticAnalysis.wl>.
- [3] *SeriesOperations.wl*, pinned source. <https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/src/Kernel/SeriesOperations.wl>.
- [4] *NativeCompatibility.wl*, pinned source. <https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/src/Kernel/NativeCompatibility.wl>.
- [5] *InverseCertificates.wl*, pinned source. <https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/src/Kernel/InverseCertificates.wl>.
- [6] *DirichletSpecialFunctions.wl*, pinned source. <https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/src/Kernel/DirichletSpecialFunctions.wl>.
- [7] *NativeSpecialFunctions.wl*, pinned source. <https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/src/Kernel/NativeSpecialFunctions.wl>.
- [8] *InverseFunctionExpressions.wl*, pinned source. <https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/src/Kernel/InverseFunctionExpressions.wl>.
- [9] *RefinementRequests.wl*, pinned source. <https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/src/Kernel/RefinementRequests.wl>.
- [10] *Code review implementation status*, pinned maintained finding register. https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/docs/development/CODE_REVIEW_STATUS.md.
- [11] *Code review reports: wave 1*, pinned index. <https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/external-reports/code-review/wave-1/README.md>.
- [12] *Code review reports: wave 2*, pinned index. <https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/external-reports/code-review/wave-2/README.md>.
- [13] *Review 4 finding ledger*, pinned archive. <https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/external-reports/code-review/wave-1/code-review-4/evidence/findings.json>.
- [14] *Review 6 finding ledger*, pinned archive. <https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/external-reports/code-review/wave-1/code-review-6/evidence/findings.csv>.
- [15] *Review 11 delta ledger*, pinned archive. <https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/external-reports/code-review/wave-2/code-review-11/evidence/findings-delta.json>.

- [16] *Native expansion result contracts*, pinned documentation. https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/docs/development/NATIVE_RESULT_CONTRACTS.md.
- [17] *Native expansion compatibility: required coverage and implementation plan*, pinned documentation. https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e35c6763d4/docs/development/NATIVE_COMPATIBILITY.md.
- [18] Wolfram Research, *Series*, official Wolfram Language documentation, accessed September 9, 2026 (Pacific time). <https://reference.wolfram.com/language/ref/Series.html>.
- [19] Wolfram Research, *SeriesData*, official documentation, accessed on the same date. <https://reference.wolfram.com/language/ref/SeriesData.html>.
- [20] Wolfram Research, *InverseSeries*, official documentation, accessed on the same date. <https://reference.wolfram.com/language/ref/InverseSeries.html>.
- [21] Wolfram Research, *ComposeSeries*, official documentation, accessed on the same date. <https://reference.wolfram.com/language/ref/ComposeSeries.html>.
- [22] Wolfram Research, *Asymptotic*, official documentation, accessed on the same date. <https://reference.wolfram.com/language/ref/Asymptotic.html>.
- [23] Wolfram Research, *AsymptoticSolve*, official documentation, accessed on the same date. <https://reference.wolfram.com/language/ref/AsymptoticSolve.html>.
- [24] Wolfram Research, *AsymptoticLess*, official documentation, accessed on the same date. <https://reference.wolfram.com/language/ref/AsymptoticLess.html>.
- [25] Wolfram Research, *FunctionAnalytic*, official documentation, accessed on the same date. <https://reference.wolfram.com/language/ref/FunctionAnalytic.html>.